Manual avanzado de GitHub

Unidad 2. Entornos de Desarrollo, 1.º DAW

Índice

Antes de empezar **2**

Parte 1. Repaso de lo básico **3**
1.1. Git y GitHub en cuatro frases . . . 3
1.2. Los cuatro términos que hay que tener claros . . . 3
1.3. Qué guarda exactamente un commit . . . 3
1.4. Volver a una versión anterior . . . 3
1.5. revert y reset: la diferencia que hay que entender . . . 4
1.6. Lo único que no se puede recuperar . . . 6

Parte 2. Ramas **7**
2.1. El problema que resuelven . . . 7
2.2. Qué es exactamente una rama . . . 7
2.3. De dónde viene lo de «trunk» . . . 7
2.4. Cómo ramifica la gente de verdad . . . 8
2.5. Cómo se nombran las ramas . . . 9
2.6. Crear una rama y cambiar a ella . . . 9
2.7. Subir la rama a GitHub . . . 9

Parte 3. Juntar el trabajo **10**
3.1. Qué es un merge . . . 10
3.2. Qué es un pull request . . . 10
3.3. Abrir un pull request paso a paso . . . 10
3.4. Revisar un pull request . . . 10
3.5. Fusionar: las tres opciones del botón verde . . . 11
3.6. Después de fusionar . . . 11

Parte 4. Conflictos **12**
4.1. Por qué aparecen . . . 12
4.2. Qué vas a ver . . . 12
4.3. Resolverlo . . . 12
4.4. Resolverlo desde la web . . . 13
4.5. Cómo tener menos conflictos . . . 13

Parte 5. Usar código de otra gente **14**
5.1. Las cuatro formas, y cuándo usar cada una . . . 14
5.2. Download ZIP . . . 14
5.3. Clonar . . . 14
5.4. Releases: la versión que de verdad funciona . . . 14
5.5. Fork: cuando quieres cambiar algo que no es tuyo . . . 15

5.6. Mantener el fork al día . . . 15
5.7. La licencia: antes de usar nada . . . 15
5.8. Contribuir a un proyecto ajeno . . . 16

Parte 6. Las herramientas del repositorio **17**
6.1. El fichero .gitignore . . . 17
6.2. Publicar una versión . . . 17
6.3. Projects: el tablero . . . 17
6.4. Proteger la rama main . . . 17
6.5. Actions, por encima . . . 18
6.6. Dos cosas pequeñas que se agradecen . . . 18

Parte 7. Lo mismo, desde la terminal **19**
7.1. Tabla de equivalencias . . . 19
7.2. El historial en una pantalla . . . 19
7.3. Deshacer cosas . . . 19
7.4. Guardar cambios a medias: stash . . . 20
7.5. SSH en lugar de contraseñas . . . 20

Cuando algo va mal **21**

Glosario **21**

Resumen en once líneas **22**

Antes de empezar

Este manual continúa donde acaba el básico. La Parte 1 repasa lo esencial en dos páginas, por si hace falta refrescarlo; a partir de la Parte 2 entra lo que de verdad diferencia a alguien que «ha usado GitHub» de alguien que sabe trabajar con él: **ramas, juntar el trabajo de varias personas, resolver conflictos y usar código de otros**.

Casi todo se puede hacer con el ratón, y así se explica. Los comandos equivalentes están en la Parte 7, para quien quiera.

Parte	Qué vas a conseguir
1	Repaso: qué es Git, qué es GitHub y cómo volver a una versión anterior
2	Entender qué es una rama y por qué `main` se llamaba `trunk`
3	Juntar ramas: merge y pull requests
4	Salir vivo de un conflicto
5	Descargar y usar código de otra gente sin meter la pata
6	Las herramientas del repositorio: versiones, tableros, protecciones
7	Los comandos de terminal, por si te hacen falta

Parte 1. Repaso de lo básico

1.1. Git y GitHub en cuatro frases

Git es un programa que se instala en tu ordenador. Su trabajo es vigilar una carpeta y guardar, cada vez que tú se lo pidas, **una copia completa de cómo estaba todo en ese momento**. Funciona sin internet y no necesita a nadie más.

GitHub es una página web donde se guarda una copia de ese proyecto, con todo su historial, para tenerlo a salvo, verlo desde cualquier ordenador y compartirlo con otras personas.

Dicho de otro modo: **Git es la herramienta; GitHub es el sitio donde se deja el resultado.** Puedes usar Git sin GitHub perfectamente. Lo que no puedes es usar GitHub sin Git, porque GitHub no es más que un almacén de repositorios de Git con un montón de cosas montadas alrededor.

Hay más sitios que hacen lo mismo (GitLab, Bitbucket, Codeberg) y todos usan Git por debajo.

1.2. Los cuatro términos que hay que tener claros

Palabra	Qué es
Repositorio	La carpeta del proyecto **más su historial completo**. En tu ordenador, el historial vive en una subcarpeta oculta llamada `.git`
Commit	Una foto guardada del proyecto entero, con fecha, autor y un mensaje que explica el cambio
Push	Mandar tus commits a GitHub
Pull	Traerte de GitHub los commits que no tengas

Y la confusión número uno, que conviene repetir: **commit no es subir**. El commit guarda la foto en tu ordenador; el push es lo que la manda a GitHub. Puedes hacer diez commits en un tren sin cobertura.

1.3. Qué guarda exactamente un commit

Esto es lo que hace que todo lo demás funcione, así que merece un párrafo.

Un commit **no guarda «lo que ha cambiado»: guarda el proyecto entero** tal y como estaba en ese instante. Git es listo y no duplica por dentro los ficheros que no han cambiado, pero desde fuera la idea es esa: cada commit es una fotografía completa.

Cada commit tiene además:

- Un **identificador** único, un churro de 40 letras y números (`a3f91c2...`) del que normalmente solo se usan los siete primeros.
- Un **puntero a su commit anterior**. Por eso el historial es una cadena: desde el último se llega caminando hasta el primero.

1.4. Volver a una versión anterior

Sí: **puedes volver a cualquier commit, siempre**. Mientras esté confirmado, está ahí. Pero «volver» puede significar cuatro cosas distintas, y conviene no mezclarlas.

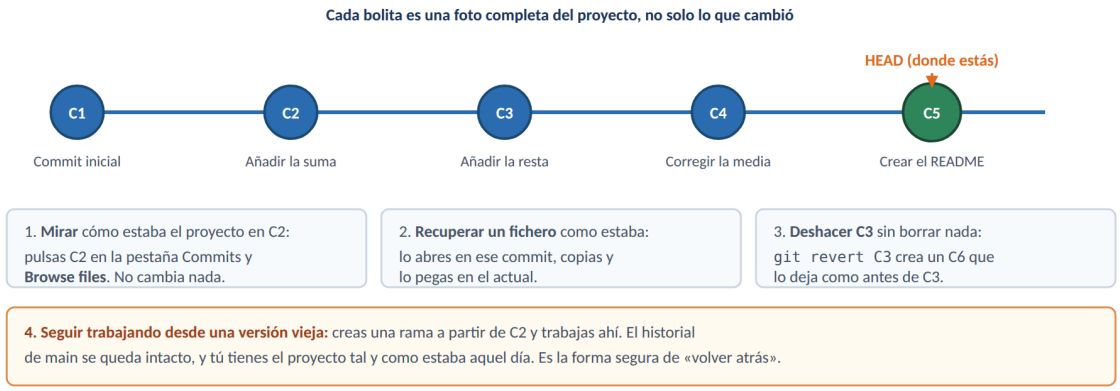


Figura 1: *Las cuatro formas de volver atrás en el historial*

Lo que quieres	Dónde se hace	Comando
Mirar cómo estaba el proyecto	Pestaña **Commits** > pulsas uno > botón **Browse files**	`git switch --detach a3f91c2`
Recuperar un fichero suelto	Abres el fichero en ese commit y copias su contenido	`git restore --source=a3f91c2 fichero.py`
Deshacer un commit concreto	Pestaña Commits > el commit > menú **Revert**	`git revert a3f91c2`
Seguir trabajando desde ahí	Crear una rama a partir de ese commit	`git switch -c rescate a3f91c2`

Idea clave

La diferencia importante. *Mirar* y *recuperar un fichero* no cambian nada: son seguros siempre. *Revert* tampoco borra nada: **crea un commit nuevo** que deja el código como estaba antes de aquel. El historial sigue entero, y se ve que hubo un error y que se corrigió. Es lo que se usa cuando el commit ya está subido.
Y si lo que quieres es retomar el proyecto desde un punto antiguo, **lo correcto es crear una rama ahí**, no borrar lo de después.

1.5. revert y reset: la diferencia que hay que entender

Son las dos formas de «deshacer», y no se parecen en nada. Esta es probablemente la distinción más importante de todo el manual.

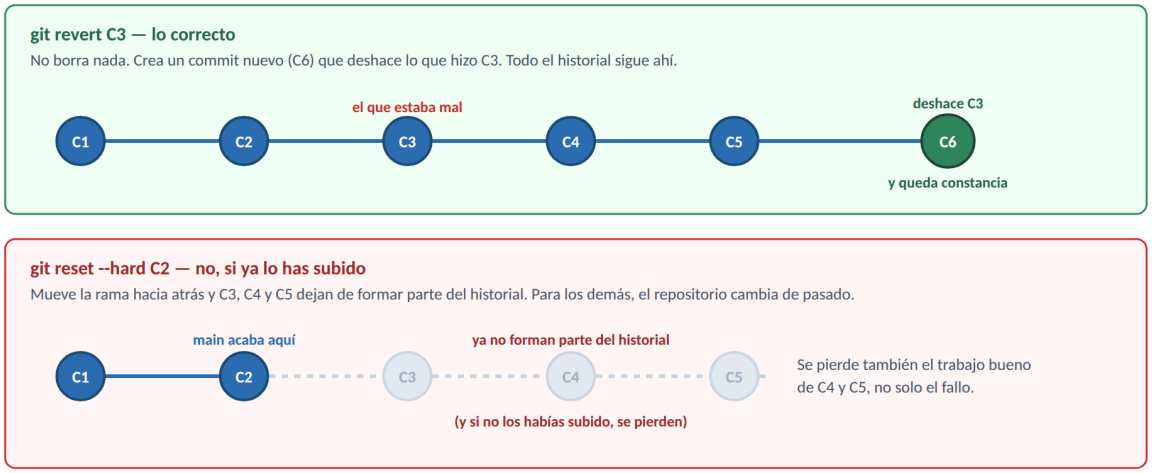


Figura 2: *Lo mismo resuelto bien y resuelto mal*

	`git revert C3`	`git reset --hard C2`
Qué hace	Crea un commit nuevo que deshace aquel	Mueve la rama hacia atrás
El historial	Queda entero. Se ve el error y la corrección	C3, C4 y C5 salen del historial
¿Se puede subir?	Sí, es un commit normal	Solo con `push --force`, que es el problema
Para los demás	No pasa nada	El repositorio «cambia de pasado»
Precisión	Deshace **solo** ese commit	Se lleva por delante **todo** lo posterior

Ojo

Por qué `reset --hard` es un problema en un repositorio compartido.
Tu compañero tiene C3, C4 y C5 en su ordenador. Tú los borras del historial y fuerzas la subida. A partir de ese momento, el repositorio de GitHub y el suyo cuentan dos historias distintas del mismo proyecto: cuando él haga `pull`, Git no sabrá reconciliarlas y le saldrán errores raros, o peor, volverá a subir los commits que tú acabas de borrar.
Y hay un detalle que se olvida: **`reset --hard C2` no borra solo el commit malo**. Se lleva también C4 y C5, que estaban bien. Si lo que quieres es deshacer C3, usa `revert C3`, que es **cirugía**; `reset` es una amputación.

Entonces, ¿cuándo sirve `reset`? Cuando el commit **todavía no ha salido de tu ordenador**.

Situación	Qué usar
El commit ya está en GitHub	`git revert` **siempre**
El commit está solo en tu máquina y quieres rehacerlo	`git reset --soft HEAD~1`: deshace el commit y **te devuelve los cambios** para volver a confirmarlos
El commit está solo en tu máquina y quieres tirarlo todo	`git reset --hard HEAD~1`. Lo que había ahí se pierde

Tres reglas y ya:

1. **`revert` para lo que ya has subido.** Siempre.

2. **`reset --soft` para lo tuyo sin subir.** Guarda los cambios.
3. **`reset --hard` y `push --force`: no**, en nada que sea compartido. No es que esté mal visto; es que le rompe el repositorio a los demás.

Idea clave

Si la has liado con un `reset --hard` local, aún queda una bala: `git reflog` lista todos los sitios donde ha estado tu rama en los últimos días, con sus identificadores. Buscas el commit de antes del desastre y creas una rama ahí:

```
git reflog
git switch -c rescate a3f91c2
```

Funciona porque Git tarda semanas en limpiar los commits que ya no apunta nadie. Pero es un rescate, no un método: no trabajes contando con él.

1.6. Lo único que no se puede recuperar

Qué	Por qué
Lo que **nunca llegaste a confirmar**	Si no hiciste commit, Git no se enteró de que existía
Lo que hiciste commit pero **no push**, si se rompe el portátil	Esa foto solo estaba en tu disco
Lo que borraste con `git reset --hard` **y no habías subido**	Se puede rescatar un tiempo con `git reflog`, pero no cuentes con ello

De ahí las dos costumbres de siempre: **commits pequeños y frecuentes**, y **push al terminar de trabajar**.

Ojo

Un aviso para más adelante. En la Parte 3 se explica la opción *Squash and merge*, que aplasta todos los commits de una rama en uno solo al fusionarla. Es cómoda y deja el historial de `main` muy limpio, pero a cambio **los commits intermedios de esa rama dejan de estar en `main`**: solo quedan en la rama, y si la borras, desaparecen de la vista.

Para trabajos de clase es perfectamente aceptable. Solo tenlo presente: no todo lo que haces queda en `main` para siempre si usas squash.

Parte 2. Ramas

2.1. El problema que resuelven

Imagina que tu proyecto funciona y quieres añadir una pantalla de login. Vas a tardar tres días. Durante esos tres días, el proyecto está roto: a medias.

Si trabajas en `main` y sucede cualquiera de estas cosas, tienes un problema:

- El profesor pide ver el proyecto hoy.
- Tu compañero necesita el código para su parte.
- Descubres un fallo urgente que hay que corregir ya.

Una **rama** es una línea de trabajo paralela. Te llevas el proyecto a un lado, lo revientas todo lo que quieras, y **`main` sigue intacta y funcionando**. Cuando lo tienes terminado, juntas las dos.

Idea clave

La regla de oro: `main` siempre funciona. Todo lo que esté a medias vive en una rama.

2.2. Qué es exactamente una rama

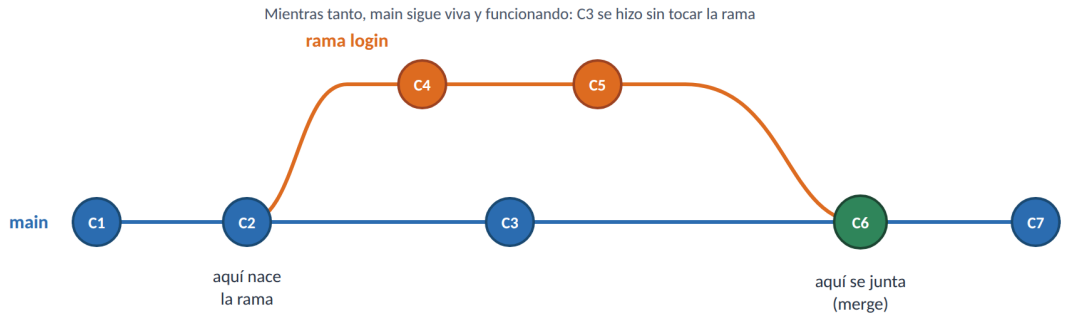


Figura 3: *Una rama sale de un commit, avanza por su cuenta y vuelve a juntarse*

Mira el dibujo con calma:

- `main` va de C1 a C7. Es la línea principal.
- En C2 alguien crea la rama `login`. A partir de ahí, C4 y C5 **solo existen en esa rama**.
- Mientras tanto, en `main` se hizo C3. Las dos líneas avanzan sin estorbarse.
- En C6 se juntan: eso es el **merge**. A partir de ahí, `main` tiene todo.

Técnicamente, **una rama en Git no es una copia de nada**: es solo una etiqueta que apunta a un commit, y se mueve sola hacia delante según vas confirmando. Por eso crear una rama es instantáneo, aunque el proyecto pese un giga.

2.3. De dónde viene lo de «trunk»

Esta palabra te la vas a encontrar en documentación, en ofertas de trabajo y en los apuntes de sistemas de control de versiones. Conviene entenderla.



Figura 4: *En SVN eran carpetas; en Git son etiquetas*

Trunk significa *tronco*. Viene de los sistemas anteriores a Git, sobre todo **Subversion (SVN)** y **CVS**, que organizaban el repositorio en tres carpetas de verdad:

Carpeta	Qué guardaba
`trunk/`	La línea principal de desarrollo. El tronco
`branches/`	Una copia completa del proyecto por cada línea paralela
`tags/`	Una copia congelada de cada versión publicada (v1.0, v1.1…)

En aquellos sistemas, **crear una rama era copiar el proyecto entero** a otra carpeta del servidor. Era lento, ocupaba, y juntarlo después era tan doloroso que la mayoría de equipos simplemente no ramificaba: todo el mundo escribía en el `trunk`.

Git cambió eso. Las ramas pesan unos pocos bytes, se crean en un segundo y juntarlas es normal. Por eso en Git ramificamos a todas horas.

Ojo

Equivalencias que tienes que tener claras:

En SVN / CVS	En Git / GitHub
`trunk`	La rama `main` (antes `master`)
Una carpeta en `branches/`	Una rama cualquiera: `login`, `arreglo-media`…
Una carpeta en `tags/`	Una **etiqueta** (*tag*), normalmente publicada como *release*
Copiar el proyecto para ramificar	Crear una etiqueta que apunta a un commit

`main` **es** el trunk de toda la vida. Solo le han cambiado el nombre.

Sobre el nombre: hasta 2020 la rama principal se llamaba `master` por defecto. GitHub la cambió a `main`. Te vas a encontrar las dos: en repositorios antiguos, `master`; en los nuevos, `main`. Hacen exactamente lo mismo.

2.4. Cómo ramifica la gente de verdad

No hay una única forma. Estas son las dos que te vas a encontrar:

Modelo	Cómo funciona	Dónde se usa
Trunk-based	Ramas muy cortas (horas o un día) que se juntan a `main` enseguida	Lo más extendido hoy. Es lo que vamos a hacer en clase
Git Flow	Ramas fijas `main` y `develop`, más ramas de `feature`, `release` y `hotfix`	Proyectos con versiones que se publican e instalan

Para lo que hacemos aquí, **trunk-based**: una rama por tarea, corta, y a `main` en cuanto funcione. Las ramas que viven dos semanas son las que acaban dando conflictos horribles.

2.5. Cómo se nombran las ramas

Sin espacios, sin acentos, en minúsculas y **diciendo qué hay dentro**:

- Bien: `login`, `arreglo-media`, `formulario-contacto`, `fix/division-por-cero`
- Mal: `rama1`, `pruebas`, `nueva`, `juan`

2.6. Crear una rama y cambiar a ella

Desde VS Code (lo habitual). En la **barra azul de abajo del todo**, a la izquierda, pone el nombre de la rama actual (`main`). Púlsalo:

1. Se abre una lista arriba con las ramas que hay.
2. Elige **Create new branch...**
3. Escribe el nombre y pulsa Intro.

Listo. La barra de abajo ya pone el nombre nuevo, y **todo lo que hagas a partir de ahora va a esa rama**. Para volver a `main`, el mismo botón y la eliges de la lista.

Desde la web de GitHub. En la página del repositorio, el desplegable que pone `main`: escribe un nombre que no exista y aparece la opción *Create branch: ... from main*.

Ojo

Antes de crear una rama, ponte en `main` y haz pull. Si creas la rama a partir de un `main` viejo, estás partiendo de código desactualizado y te estás buscando un conflicto.

2.7. Subir la rama a GitHub

La primera vez que haces commit en una rama nueva, VS Code no pone *Sync Changes* sino **Publish Branch**. Púlsalo: eso crea la rama también en GitHub.

A partir de ahí, *Sync Changes* normal.

Parte 3. Juntar el trabajo

3.1. Qué es un merge

Un **merge** (*fusión*) coge todos los commits de una rama y los mete en otra. Es el C6 del dibujo de antes.

Git lo hace solo, fichero por fichero y línea por línea. Solo te pregunta cuando **las dos ramas han tocado exactamente las mismas líneas**: eso es un conflicto, y va en la Parte 4.

Hay dos formas de que salga:

Tipo	Cuándo pasa	Qué queda en el historial
Fast-forward	`main` no se ha movido desde que naciste	Los commits se enganchan sin más. No hay commit de merge
Merge commit	`main` ha avanzado por su cuenta (hay un C3)	Se crea un commit extra que une las dos líneas

3.2. Qué es un pull request

Un **pull request** (PR) es **pedir permiso para juntar tu rama**, con una página de discusión delante.

El nombre engaña: no tiene nada que ver con el `pull` de traerse cambios. Es «solicitud de incorporación»: «*he hecho esto en mi rama, miradlo, y si os parece bien lo metéis en main*».

Sin pull request	Con pull request
Juntas tú y ya está	Alguien lo revisa antes
No queda explicación de por qué	Queda la descripción, los comentarios y la discusión
No se ve el resumen de los cambios	Se ven todas las líneas tocadas, agrupadas
Nadie se entera	Se avisa al equipo

Idea clave

Úsalo aunque trabajes solo. Cuesta un minuto y te obliga a releer tus propios cambios antes de meterlos. Es, además, la forma de trabajar de cualquier empresa: si llegas a unas prácticas sabiendo hacer una PR decente, se nota.

3.3. Abrir un pull request paso a paso

1. Sube tu rama (*Publish Branch* o *Sync Changes*).
2. Entra en el repositorio en GitHub. Aparece un aviso amarillo: **Compare & pull request**. Púlsalo. *(Si no sale: pestaña **Pull requests** > **New pull request**.)*
3. Arriba verás **base: main <- compare: tu-rama**. Comprueba que están bien puestas: *base* es a dónde va, *compare* es de dónde viene.
4. **Título:** qué hace, en una línea.
5. **Descripción:** qué has cambiado y cómo se prueba. Si arregla un issue, escribe `Closes #3` y GitHub lo cerrará solo al fusionar.
6. **Create pull request**.

3.4. Revisar un pull request

Esto es lo que hace quien recibe la PR, y lo que vas a hacer tú con la de tus compañeros.

Pestaña de la PR	Para qué
Conversation	La descripción y todos los comentarios
Commits	La lista de commits que entran
Files changed	**La importante.** Todas las líneas tocadas: verde añadido, rojo quitado

En *Files changed*, al pasar el ratón por una línea aparece un **+ azul**: púlsalo y comentas **esa línea concreta**. Es muchísimo más útil que decir «hay algo raro en el fichero».

Cuando terminas, botón **Review changes** arriba a la derecha:

- **Comment**: opino, pero no bloqueo.
- **Approve**: por mí, adelante.
- **Request changes**: así no; corrige esto.

3.5. Fusionar: las tres opciones del botón verde

Al fusionar, GitHub te deja elegir **cómo** entran los commits. El desplegable del botón verde:

Opción	Qué hace	Cuándo usarla
Create a merge commit	Entran todos tus commits, más uno extra que los une	Por defecto. Conserva todo el historial
Squash and merge	Aplasta toda la rama en **un solo commit**	Si has hecho ocho commits del tipo «pruebas», «ya va»
Rebase and merge	Reescribe tus commits al final de `main`, en línea recta	Historial limpio, pero cambia los identificadores

Para clase: **Squash and merge** casi siempre. Deja un historial de `main` donde cada línea es una tarea terminada y se lee de un vistazo.

3.6. Después de fusionar

GitHub te ofrece **Delete branch**. Dile que sí: la rama ya ha cumplido y sus commits están en `main`. Si la necesitas, se puede recuperar.

Y en tu ordenador: ponte en `main` y haz **pull**, para traerte lo que acabas de fusionar.

Parte 4. Conflictos

4.1. Por qué aparecen

Un **conflicto** aparece cuando dos ramas han cambiado **la misma línea del mismo fichero** de forma distinta. Git sabe juntar cambios en sitios diferentes; lo que no puede es decidir cuál de las dos versiones de una línea quieres.

No es un error ni has roto nada. Es Git preguntándote.

4.2. Qué vas a ver

VS Code abre el fichero con los dos trozos marcados y unos botones azules encima:

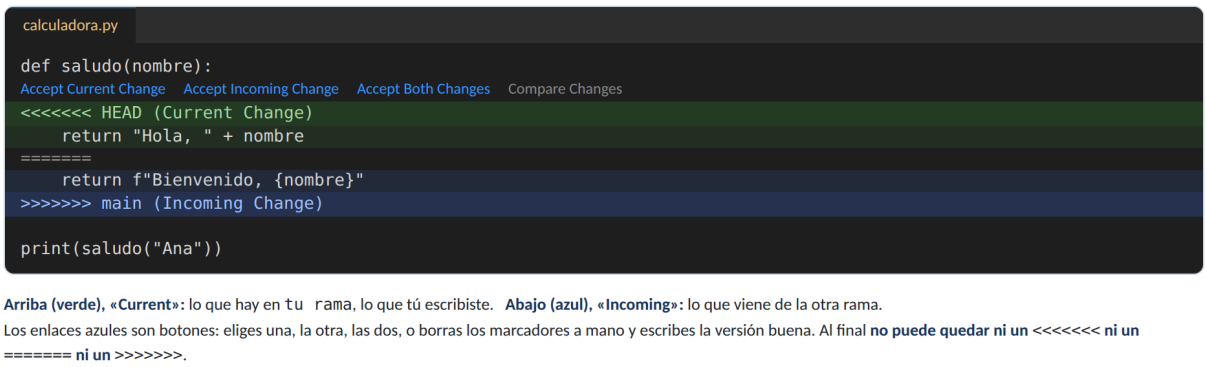


Figura 5: *Un conflicto abierto en VS Code*

En texto plano, los marcadores son estos:

```
<<<<<<< HEAD
    return "Hola, " + nombre
=======
    return f"Bienvenido, {nombre}"
>>>>>>> main
```

Marcador	Qué significa
`<<<<<<< HEAD`	Empieza **tu versión**, la de la rama en la que estás
`=======`	Separador
`>>>>>>> main`	Termina la versión **que viene de la otra rama**

4.3. Resolverlo

Tienes dos caminos, y los dos acaban igual.

Con los botones (lo normal). Encima del conflicto, VS Code pone cuatro enlaces:

Botón	Qué deja
Accept Current Change	Solo tu versión
Accept Incoming Change	Solo la de la otra rama
Accept Both Changes	Las dos, una detrás de otra
Compare Changes	Las abre lado a lado para verlas mejor

A mano. Borra las tres líneas de marcadores y deja escrito el código que tiene que quedar. Muchas veces la versión buena no es ninguna de las dos, sino una mezcla.

Después, siempre:

1. Comprueba que **no queda ni un marcador** en el fichero. Busca `<<<<` con `Ctrl+F`.
2. **Ejecuta el programa.** Un conflicto mal resuelto compila y da un resultado equivocado.
3. Prepara el fichero (**+**) y haz commit. VS Code ya propone un mensaje tipo `Merge branch 'main' into login`; déjalo.

4.4. Resolverlo desde la web

Si el conflicto sale al fusionar una PR, GitHub avisa con *This branch has conflicts* y un botón **Resolve conflicts** que abre un editor en el navegador con los mismos marcadores. Sirve para conflictos de una o dos líneas. Para algo más gordo, mejor en tu ordenador: ahí puedes probar que funciona.

4.5. Cómo tener menos conflictos

- **Pull antes de empezar, siempre.**
- **Ramas cortas.** Una rama de dos semanas es una bomba de relojería.
- **Repartiros los ficheros**, no las líneas. Que cada uno trabaje en su zona.
- Juntad `main` a vuestra rama de vez en cuando, sin esperar al final.

Ojo

No borres el repositorio y lo clones otra vez. Es la reacción típica cuando aparece el primer conflicto, y se pierde trabajo. Un conflicto se resuelve en dos minutos cuando sabes qué estás mirando.

Parte 5. Usar código de otra gente

Casi todo el software que usas está publicado en GitHub. Saber traértelo es media asignatura.

5.1. Las cuatro formas, y cuándo usar cada una

Forma	Qué te llevas	Cuándo
Download ZIP	Los ficheros, **sin historial**	Solo quieres mirarlo o probarlo una vez
Release	Una versión concreta, ya probada	Vas a **usar** el programa, no a tocarlo
Clone	El proyecto entero con todo su historial	Vas a trabajar con él
Fork	Una copia **tuya en GitHub**, y luego la clonas	Vas a cambiarlo o a proponer mejoras

5.2. Download ZIP

Botón verde **Code** > **Download ZIP**. No hace falta ni tener cuenta.

Lo que bajas es una carpeta normal: **no es un repositorio**, no tiene historial y no se puede actualizar. Si luego quieres trabajar en serio, tendrás que clonarlo otra vez.

5.3. Clonar

Botón verde **Code** > pestaña **HTTPS** > copiar la dirección. En VS Code, `Ctrl+Mayús+P` > **Git: Clone** > pegar.

Puedes clonar cualquier repositorio público aunque no sea tuyo. Lo tendrás entero, con su historial: podrás leerlo, ejecutarlo y trastear. Lo que **no** podrás es hacer push, porque no tienes permiso de escritura. Para eso está el fork.

5.4. Releases: la versión que de verdad funciona

En la columna derecha del repositorio hay un apartado **Releases**. Ahí el autor publica versiones cerradas y numeradas (`v2.1.0`), con sus notas de cambios y, a veces, el programa ya compilado listo para instalar.

Qué bajas	Qué es
Source code (zip)	El código tal como estaba en esa versión
Los demás ficheros	El programa ya montado: `.exe`, `.deb`, `.jar`, `.AppImage`…

Idea clave

`main` no es la versión estable. `main` es lo último que han tocado los desarrolladores y puede estar a medias. Si vas a usar un programa, **baja una release**. Si vas a estudiar el código, clona `main`.

Una release se apoya en una **etiqueta** (*tag*): el equivalente exacto de las carpetas `tags/` de SVN que vimos en la Parte 2.

5.5. Fork: cuando quieres cambiar algo que no es tuyo



Figura 6: *El recorrido completo de un fork*

Un **fork** es una copia del repositorio de otra persona **dentro de tu cuenta**. Es tuya: ahí sí puedes hacer push.

El recorrido completo, que es el estándar de todo el software libre:

1. En el repositorio original, botón **Fork** (arriba a la derecha) > **Create fork**.
2. Ya tienes `github.com/tuusuario/proyecto`. **Clona ese**, no el original.
3. Crea una rama para tu cambio.
4. Trabaja, commit y push: va a tu fork.
5. En tu fork aparece el aviso **Contribute** > **Open pull request**.
6. La PR se abre **contra el repositorio original**. Su autor decide si la acepta.

Al repositorio original se le llama **upstream** («aguas arriba»).

5.6. Mantener el fork al día

Tu fork no se actualiza solo. Si el proyecto original avanza, el tuyo se queda atrás y GitHub te lo dice: «*This branch is N commits behind*».

Al lado hay un botón **Sync fork** > **Update branch**. Un clic y ya está.

Hazlo **antes** de empezar cada cambio, por lo mismo de siempre.

5.7. La licencia: antes de usar nada

Esto no es un detalle burocrático. Si no hay licencia, **por defecto no tienes derecho a usar ese código**, aunque esté publicado.

Licencia	Qué te deja hacer	Qué te obliga
MIT, **BSD**	Prácticamente todo, incluso vender	Mantener el aviso de autoría
Apache 2.0	Como MIT	Autoría, y avisar de qué has cambiado
GPL	Usarlo y modificarlo	**Publicar tu proyecto también con GPL**
Sin licencia	Nada. Solo mirarlo	—

La licencia está en el fichero `LICENSE` y GitHub la enseña en la columna derecha.

Ojo

En las prácticas de clase, cita siempre de dónde has sacado el código. Una línea en el README con el enlace. Copiar sin citar es plagio aquí y en cualquier sitio; citando, es reutilizar, que es exactamente lo que hace todo el mundo en esta profesión.

5.8. Contribuir a un proyecto ajeno

Si te animas, el orden es siempre este:

1. Lee el **README** y el fichero **CONTRIBUTING.md** si lo hay. Ahí ponen sus normas.
2. Mira los **Issues**, filtrando por la etiqueta **good first issue**: son los que el propio equipo marca como fáciles para gente que llega nueva.
3. Comenta en el issue que te lo quedas, para que no lo haga otro a la vez.
4. Fork, rama, cambio, pull request.

Parte 6. Las herramientas del repositorio

6.1. El fichero .gitignore

Es la lista de lo que Git debe ignorar. Una ruta o un patrón por línea:

```
__pycache__/
*.log
.env
node_modules/
.vscode/
```

Patrón	Qué ignora
`carpeta/`	Esa carpeta entera
`*.log`	Todos los ficheros con esa extensión
`.env`	Ese fichero concreto (el típico de las contraseñas)
`!importante.log`	Excepción: este sí se sube

En `github.com/github/gitignore` tienes plantillas hechas para cada lenguaje.

Ojo

El `.gitignore` solo vale para ficheros que todavía no has subido. Si ya subiste algo una vez, añadirlo al `.gitignore` no lo borra del historial: sigue ahí y se puede recuperar. Por eso las contraseñas hay que cambiarlas, no borrarlas.

6.2. Publicar una versión

Cuando una entrega está terminada, márcala:

1. Columna derecha > **Releases** > **Create a new release**.
2. **Choose a tag** > escribe `v1.0` > *Create new tag on publish*.
3. Título y notas: qué tiene esta versión.
4. Puedes **arrastrar ficheros** al recuadro de abajo (un ZIP, un ejecutable).
5. **Publish release**.

Las versiones se numeran `MAYOR.MENOR.PARCHE` (`2.4.1`): el último número para correcciones, el del medio para cosas nuevas, el primero cuando rompes la compatibilidad.

6.3. Projects: el tablero

La pestaña **Projects** monta un tablero de tarjetas (*To do*, *In progress*, *Done*) donde cada tarjeta puede ser un issue o una PR. Se arrastran con el ratón y, si la tarjeta es un issue, **se mueve sola a Done** cuando lo cierras.

Para un trabajo en grupo de tres o cuatro personas, es suficiente y no hay que aprender nada.

6.4. Proteger la rama main

En equipo, lo más útil que puedes activar. **Settings** > **Branches** > **Add branch ruleset** (o *Add rule*), para `main`:

Opción	Qué consigue
Require a pull request before merging	Nadie puede subir directamente a `main`
Require approvals	Hace falta que alguien apruebe la PR
Require status checks to pass	Si los tests fallan, no se puede fusionar

Con lo primero ya evitas el accidente más típico del curso: subir a `main` sin querer.

6.5. Actions, por encima

La pestaña **Actions** sirve para que GitHub **ejecute cosas solo** cada vez que subes código: pasar los tests, comprobar el estilo, generar la documentación, desplegar la web.

Se configura con un fichero `.github/workflows/algo.yml`. Esto es **integración continua** y se ve en condiciones en la Unidad 4; de momento basta con que sepas que la pestaña está ahí y para qué es.

6.6. Dos cosas pequeñas que se agradecen

Plantillas de issue. Si creas `.github/ISSUE_TEMPLATE/fallo.md`, ese texto saldrá relleno cada vez que alguien abra un issue. Sirve para que no te manden «no funciona» y ya está.

La wiki. Pestaña **Wiki**: páginas en Markdown para documentación larga, cuando el README se queda corto.

Parte 7. Lo mismo, desde la terminal

Todo lo anterior se hace con el ratón. Pero los comandos aparecen en toda la documentación del mundo, y hay cosas que son más rápidas escribiéndolas.

7.1. Tabla de equivalencias

Lo que quieres	En VS Code	Comando
Ver cómo está todo	El panel de cambios	`git status`
Crear una rama y cambiar a ella	Barra de abajo > Create new branch	`git switch -c login`
Cambiar de rama	Barra de abajo > elegirla	`git switch main`
Ver las ramas	Barra de abajo	`git branch`
Preparar un fichero	El **+**	`git add fichero`
Preparar todo	El **+** de la cabecera	`git add .`
Confirmar	Botón *Commit*	`git commit -m "Mensaje"`
Subir	*Sync Changes*	`git push`
Subir una rama nueva	*Publish Branch*	`git push -u origin login`
Bajar	*Sync Changes*	`git pull`
Juntar otra rama con la tuya	Command Palette > *Git: Merge*	`git merge login`
Ver el historial	Pestaña *Timeline*	`git log --oneline --graph --all`

7.2. El historial en una pantalla

```
git log --oneline --graph --all --decorate
```

Dibuja el árbol de ramas en la terminal, parecido al diagrama de la Parte 2. Es el comando que más se agradece cuando te pierdes.

7.3. Deshacer cosas

Por orden, de lo inofensivo a lo peligroso:

Situación	Comando	Qué hace
He tocado un fichero y quiero dejarlo como estaba	`git restore fichero`	Descarta los cambios **no confirmados**
Lo preparé sin querer	`git restore --staged fichero`	Lo quita de preparados, sin perder el cambio
Un commit ya subido está mal	`git revert <id>`	Crea un commit nuevo que deshace aquel. **Seguro**
El último commit, aún sin subir, está mal	`git reset --soft HEAD~1`	Deshace el commit y te devuelve los cambios

Ojo

`git revert` sí, `git reset --hard` no. Si el commit ya está en GitHub, deshacerlo reescribiendo el historial rompe el repositorio a todos los demás. `revert` deja constancia y no rompe nada.
Y `git push --force` no se usa. Nunca, en un repositorio compartido. El porqué, con dibujo, está en el apartado **1.5**.

Y el rescate, por si acaso: `git reflog` lista dónde ha estado tu rama estos días, con los identificadores, aunque los hayas sacado del historial. De ahí se puede crear una rama y recuperar lo que parecía perdido (apartado 1.5).

7.4. Guardar cambios a medias: stash

Estás a medias, y tienes que cambiar de rama ya:

```
git stash          # guarda los cambios en un cajón y deja el proyecto limpio
git stash pop      # los recupera donde los dejaste
```

En VS Code está en el menú de los tres puntos del panel de cambios: *Stash*.

7.5. SSH en lugar de contraseñas

Si te cansas de autenticarte, puedes usar una llave SSH. Una vez montada, no vuelve a pedirte nada:

```
ssh-keygen -t ed25519 -C "tucorreo@ejemplo.com"
```

Pulsa Intro a todo. Luego copia el contenido de `~/.ssh/id_ed25519.pub` (el fichero **.pub**, el otro no se enseña jamás) y pégalo en GitHub: **Settings** > **SSH and GPG keys** > **New SSH key**.

A partir de ahí, clona usando la pestaña **SSH** del botón verde en vez de HTTPS.

Cuando algo va mal

Problema	Qué pasa	Solución
He trabajado en `main` sin querer	Los cambios están en la rama equivocada	Crea la rama ahora: `git switch -c login`. Los cambios sin confirmar se van contigo
«This branch has conflicts»	Las dos ramas tocaron las mismas líneas	Parte 4
«Updates were rejected»	Hay commits en GitHub que no tienes	`pull` y vuelve a intentarlo
Mi fork está «N commits behind»	El original ha avanzado	**Sync fork** > *Update branch*
No me deja hacer push al repositorio de otro	No eres colaborador	Haz un **fork** y una PR
He fusionado una PR que no debía	Está en `main`	`git revert` del commit de merge. No reescribas el historial
He hecho `reset --hard` y he perdido commits	Siguen en el disco unos días	`git reflog`, busca el identificador y `git switch -c rescate <id>`
Un compañero ha hecho `push --force` y me falla el pull	Vuestros historiales ya no coinciden	Guarda tu trabajo aparte, clona otra vez y vuelve a aplicarlo. Y hablad
He subido `node_modules` o `__pycache__`	Falta el `.gitignore`	Añádelo, borra la carpeta del repositorio y haz commit
He perdido una rama que borré	No está perdida	En la PR cerrada, botón **Restore branch**

Glosario

Palabra	Qué significa
Commit	Una foto completa del proyecto, con su identificador y su mensaje
HEAD	Dónde estás ahora mismo: el commit sobre el que trabajas
Rama (*branch*)	Línea de trabajo paralela. Una etiqueta que apunta a un commit
trunk	El tronco: como se llamaba `main` en SVN y CVS
main / master	La rama principal. `master` es el nombre antiguo
Merge	Juntar una rama con otra
Pull request	Solicitud de que fusionen tu rama, con revisión por medio
Squash	Aplastar varios commits en uno solo al fusionar
Conflicto	Dos ramas cambiaron la misma línea; hay que decidir
Fork	Copia de un repositorio ajeno dentro de tu cuenta
Upstream	El repositorio original del que sale tu fork
Tag (*etiqueta*)	Un nombre fijo para un commit concreto: `v1.0`
Release	Una versión publicada, con sus notas y sus ficheros
Revert	Deshacer un commit creando otro que lo contrarresta. Seguro
Reset	Mover la rama hacia atrás. `--soft` conserva los cambios; `--hard` los tira
Reflog	El registro de por dónde ha pasado tu rama. Sirve para rescatar un `reset --hard`
Stash	Cajón temporal para cambios a medias
Actions	Tareas que GitHub ejecuta solo al subir código
Upstream / origin	`origin` es tu copia en GitHub; `upstream`, el proyecto original

Resumen en once líneas

1. Git es la herramienta, en tu ordenador. GitHub es el sitio donde dejas el resultado.
2. Cada commit es una foto completa: **se puede volver a cualquiera**. Lo que no confirmaste, no.
3. `main` siempre funciona. Todo lo demás, en una rama.
4. `main` es el `trunk` de SVN; solo le cambiaron el nombre.
5. Una rama en Git no es una copia: es una etiqueta. Por eso es gratis crearla.
6. Ramas cortas. Las largas son las que dan conflictos. Pull antes de ramificar.
7. El pull request es para que alguien lo mire antes. Úsalo aunque trabajes solo.
8. Un conflicto no es un error: es Git preguntándote cuál de las dos versiones quieres.
9. Después de resolver un conflicto, **ejecuta el programa**.
10. Para usar un programa, baja una **release**. Para estudiarlo, clónalo. Para cambiarlo, fork.
11. Sin licencia, no tienes derecho a usarlo. Y cita siempre de dónde lo sacaste.